

Everything that exists

A complete record of what has been built: every file, every table, every endpoint, every guarantee and how each one is actually enforced.
Written to be read start to finish by someone who needs to know the system without reading the code.

REPOSITORY	COMMITTS	TESTS	DATABASE	ATHLETES REGISTERED
kafriadadev_backend	6	112 passing	Live on Supabase	115

01

What KAFRIADA CORE is

An identity authority for African sport that also holds other people's money. Both halves of that sentence drive every decision in the codebase.

A person registers free and receives a **KUID** — a permanent identifier, never reused, printed on a card with a QR code. Anyone can scan that card and see a public profile. For ₦2,500 the athlete adds a photograph and a verified badge, checked by a human. Clubs register free and pay ₦15,000 for a verified club badge. Transfers between clubs are recorded but no money passes through the system. A closed-loop ledger records money arriving and has no mechanism for money leaving — that last point is not a limitation, it is what keeps KAFRIADA out of banking regulation.

The pilot is Jigawa State: 27 Local Government Areas, adults only, opening in Birnin Kudu first.

The four numbers the pilot is judged on

TARGET	NUMBER	CAN WE MEASURE IT YET?
Athletes registered	5,000	YES — BUILT
Paid conversion	4%	NEEDS STAGE 2
Active clubs	81	NEEDS CLUBS
Verification turnaround	under 24h	NEEDS STAGE 2

02 Running it

Everything below works today on the machine it was built on.

WHAT YOU NEED

Python 3.13, Node 22, `psql` — all three already installed. **Docker is not needed**, because the database is Supabase rather than a local container.

```
# Set up once
cd api
py -3.13 -m venv .venv
./venv/Scripts/python.exe -m pip install -e ".[dev]"

# Configuration — never committed, git refuses it
# api/.env holds the four database connection strings and two signing keys.
# SECRET_KEY and QR_SECRET were generated on this machine and never displayed.

# Create the four database roles (once per environment, as the admin user)
psql -v ON_ERROR_STOP=1 \
    -v app_password=... -v money_password=... \
    -v reader_password=... -v migrate_password=... \
    -f infra/bootstrap-roles-supabase.sql -d "$ADMIN_URL"

# Apply the schema
cd api && ./venv/Scripts/python.exe -m alembic upgrade head

# Run the tests
./venv/Scripts/python.exe -m pytest -m "not slow"      # 112, no database contention
./venv/Scripts/python.exe -m pytest -m db              # the 9 security proofs
./venv/Scripts/python.exe -m pytest -m "db and slow"   # the registration burst

# Start the API
./venv/Scripts/python.exe -m uvicorn kafriada.main:app --reload

# The live security demonstration — for a presentation
FORCE_COLOUR=1 ./venv/Scripts/python.exe ../scripts/demo_security.py
```

THE ONE COMMAND WORTH REMEMBERING

`demo_security.py` connects to the real database and attempts the things the system promises are impossible, showing PostgreSQL refusing each one. Eight checks, about thirty seconds, exit code 0 when everything behaves. Nothing in it is simulated.

03 The shape of the system

Four tiers. Only two of them exist so far.

TIER	STATUS	WHAT IT IS	WHAT IT HOLDS
Edge	NOT SET UP	Cloudflare — firewall, cache, Lagos presence	Nothing. Caches public profiles so a match-day scan is answered in Nigeria.
Presentation	NOT BUILT	Next.js. The browser talks only to this.	Screens. No business rules, no database credential, ever.
Domain	BUILT	FastAPI. Every rule and every database credential.	Registration, the KUID mint, the public profile, authorisation. Not reachable from the internet except the public reads and, later, the Paystack webhook.
Work	NOT BUILT	Same code, no HTTP. Background jobs.	Reconciliation, expiry, image processing, the nightly integrity check.
Data	LIVE	PostgreSQL 16 on Supabase, Frankfurt	Truth. Three schemas, four roles, four privilege levels.

WHY THE BROWSER NEVER TALKS TO THE DOMAIN TIER

The presentation tier calls the domain tier *server-side*, over a private network. That means no CORS configuration, no session token in a browser-reachable API, and the entire authenticated surface is not addressable from the internet at all. An authorisation bug on a private route is still unreachable by someone who has not already breached the presentation tier.

04 Every file, and what it does

Thirty-five source files. This is all of them.

Configuration and entry

FILE	WHAT IT DOES, AND THE REASONING INSIDE IT
settings.py	Every setting, read from the environment. Refuses to start in production with interactive docs on, DEBUG logging, a wildcard host, non-empty CORS, a Paystack test key, or the app and money roles sharing one connection. Rejects placeholder secrets and anything under 32 characters. Unknown variables are an error, because a typo'd variable name silently leaves a security setting at its default.
main.py	Builds the application. Wires middleware in a deliberate order, installs error handlers that never leak internals, mounts <code>/v1</code> . Configures structured logging with a processor that redacts any key whose name suggests a secret.
middleware.py	Two pieces. Request context gives every request an id, puts it in every log line, returns it in a header, and stamps it into audit rows — an inbound id is validated against a strict pattern first, because it ends up in logs and a caller could otherwise inject newlines. Security headers sets nosniff, frame denial, a deny-everything content policy, no-referrer, and no-store, and removes the server banner.
clock.py	Time, decided explicitly. Human-facing dates in Nigeria's timezone, machine instants in UTC. Exists because <code>date.today()</code> reads the server's clock, and the year of registration is printed permanently into the KUID.

Security primitives

FILE	WHAT IT DOES, AND THE REASONING INSIDE IT
security/passwords.py	Argon2id at OWASP minimums. Two things beyond the obvious: a semaphore caps how many hashes run at once, because argon2 is CPU-heavy by design and an uncapped registration drive would starve every other request; and a timing-equalisation hash is verified on the unknown-user branch of sign-in, so response timing cannot reveal which phone numbers are registered.
security/tokens.py	Session tokens, OTP codes, idempotency keys. Session tokens are stored as SHA-256 digests , so a leaked backup of the sessions table contains nothing replayable. OTP digests are keyed with a server-held pepper, because six digits is a million possibilities and a bare digest of that is reversible in seconds.
security/signing.py	QR signatures and the Paystack webhook check. Signatures carry a key version so the secret can be rotated without invalidating every card already printed in Jigawa. Documents plainly what the signature proves — that the QR came from us — and what it does not: that the bearer is the person shown.
api/security.py	Route authorisation, declared as an argument on each route. <code>Public()</code> requires a written reason so a deliberate choice does not look like an oversight. <code>Requires()</code> takes a permission and a <i>scope</i> , and currently fails closed — an unfinished authorisation check must refuse.

Bounded contexts

Eleven were planned; four have code so far. Each is a package with a single job, and the rule between them is that nothing reaches past another context's `service.py`.

FILE	WHAT IT DOES, AND THE REASONING INSIDE IT
identity/kuid.py	The identifier itself. Frozen type, no mutation path. Parses what people actually type off a card — lowercase, stray spaces, the en-dashes a phone keyboard substitutes — while no repair can turn one valid KUID into a different valid one. Documents that uniqueness is (state, year, serial) and the LGA segment carries none, so nobody later changes the counter on a wrong assumption.
identity/service.py	Registration and the mint. The ordering of statements <i>is</i> the design — see §7. Also <code>get_public_profile</code> , whose return type lists exactly what a stranger may see, so a field not on that class cannot reach the public page by accident.
access/phone.py	Phone normalisation — the identity anchor. Thirteen ways of writing a number reduce to one stored value, which is what makes the unique constraint meaningful. Refuses anything ambiguous rather than guessing, and refuses a foreign number rather than dropping its country code and storing the rest as Nigerian.
geography/jigawa.py	Jigawa and its 27 LGAs with their permanent three-letter codes. Validates itself at import: 27 entries, no duplicate codes, all three uppercase letters — because a duplicate would make two LGAs indistinguishable inside a KUID.
audit/service.py	The only path that writes the audit log. Joins the caller's transaction, so a rolled-back change cannot leave a record claiming it happened. Strips anything whose key suggests a secret and coerces values that will not serialise, because a formatting mistake must not be able to fail a registration.

Data access, HTTP and operations

FILE	WHAT IT DOES, AND THE REASONING INSIDE IT
db/engine.py	Three engines, three roles, three privilege levels. Which engine a code path uses <i>is</i> its authority. <code>money_transaction()</code> requires a written reason that is logged, so the connection cannot be opened absent-mindedly and a single search lists everywhere money can be touched.
api/v1/athletes.py	Four routes: register, public profile, signed QR, LGA list. Validates the client IP before it reaches an <code>inet</code> column — see §11.
migrations/0001_foundation.py	Schemas, the privilege boundary, the audit log, users, sessions, roles, permissions, scope. Written as explicit SQL rather than generated, because the grants, partial indexes and immutability triggers are exactly what a generator drops.
migrations/0002_identity_spine.py	Locations with all 27 LGAs seeded, the KUID counter, athletes, career events. Contains a check that fails the migration if Jigawa does not end up with exactly 27 LGAs.
infra/bootstrap-roles.sql	The four roles for a database we own outright.
infra/bootstrap-roles-supabase.sql	The same boundary for Supabase, minus two statements that would break Supabase's own services. Verifies role privileges rather than commanding them, because Supabase's admin user is deliberately not a superuser.
scripts/demo_security.py	The live demonstration. Eight checks against the real database.
scripts/check-no-payout-path.sh	Fails the build if <code>withdraw</code> , <code>payout</code> , <code>cash_out</code> or similar appears anywhere in the source. A paragraph in an architecture document does not survive staff turnover; this does.

05 The database

Eleven tables across three schemas. Live on Supabase, both migrations applied.

Why there are three schemas and four roles

This is the single most important structural decision in the system, so it is worth being precise. A schema is a named area of the database. A role is an account. Privileges are granted per role, per table.

ROLE	USED BY	WHAT IT MAY DO
kaf_app	Every ordinary request	Read and write <code>identity</code> and <code>ops</code> . Read <code>money</code>, never write it. No DELETE anywhere except expired sessions. Cannot UPDATE or DELETE an audit row.
kaf_money	Payment confirmation and reconciliation only	The only role that will hold INSERT on the ledger. Not yet used, because the ledger tables arrive in Stage 2.
kaf_reader	Reporting, analytics, public reads	SELECT only, and its transactions are forced read-only at the cluster level, so a mistyped report cannot modify anything even on a writable connection.
kaf_migrate	The release step only	Owns every object, holds all schema-changing power. Never held by a running process. This is the role an attacker would need, and it is not present in any running container.

WHAT THIS ACTUALLY BUYS

If someone takes over the running application, they hold `kaf_app`. With that they **cannot** write a ledger row and **cannot** delete an audit record — not because the code stops them, but because the account they are using does not have permission. That is a property of the database, provable in ten seconds, not a promise in a document.

Every table

TABLE	HOLDS	NOTABLE PROTECTIONS
ops.audit_log	Every important action: who, what, when, from where	Append-only twice over. No role holds UPDATE or DELETE, and a trigger refuses both even for the owner. Retained 7 years. Indexed by time, by subject, by actor and by request id.
ops.users	Accounts — name, phone, password hash, consent	Partial unique index on the phone number where not anonymised: this is the identity anchor. A check constraint enforces E.164 shape. Consent is a recorded row with the notice version, not a checkbox that was displayed once.
ops.sessions	Live sign-ins	Stores a SHA-256 digest of the cookie, never the cookie. Two clocks: idle expiry and absolute expiry. The only table where DELETE is granted, for the expiry sweep, with the reason recorded in the migration.
ops.roles	The 8 live roles	Each declares which scope a grant of it must name.
ops.permissions	26 permission keys	A check constraint enforces the dotted naming shape.
ops.role_permissions	Which role holds which permission	super_admin is defined by a query over all permissions, so a permission added later is included without anyone remembering to.

TABLE	HOLDS	NOTABLE PROTECTIONS
ops.user_roles	Who holds which role, and where	Carries <code>scope_kind</code> and <code>scope_id</code> . A check constraint makes a scoped role without a scope impossible. Never deleted, only revoked, so the history of who held what survives.
ops.locations	Nigeria, Jigawa, and 27 LGAs	The <code>is_live</code> flag is the entire rollout control — opening a wave is an UPDATE, not a deployment. A check constraint enforces that an LGA code is exactly three uppercase letters, because it is printed into every KUID issued there.
identity.kuid_counters	One row per (year, state)	The contended row. Incremented by a single atomic statement. No row is seeded — the first registration of a year creates its own counter, so a new year or a new state needs no migration.
identity.athletes	The athlete record and their KUID	Uniqueness stated twice : on the printed KUID, and on (state, year, serial) which is what the counter guarantees. A trigger refuses any change to the KUID or the place of registration. No DELETE — erasure means anonymisation.
identity.career_events	Registration, club moves, transfers, verification	Append-only, with the same grant-and-trigger treatment as the audit log. A mistake is corrected by adding an event, never by rewriting one.

The KUID, in detail

KA-NG-JG-BKD-2026-000123

| | | | |

└ serial, six digits, allocated per state per year

| | | |

└ year of registration, in NIGERIA's timezone

| | |

└ LGA of registration – informational only, carries no uniqueness

| |

└ state – part of the uniqueness key

|

└ country

└ KAFRIADA

Uniqueness is (state, year, serial).

It records where someone REGISTERED, not where they live. People move; this does not.

It is immutable and never reused – enforced by a database trigger, not just by code.

06 The API

Four routes live, all verified against the real database.

ROUTE	WHO	BEHAVIOUR
POST /v1/register	PUBLIC	Creates the account, mints the KUID, writes the career event and the audit row — all in one transaction. Returns the KUID, the profile address and the QR address. The same phone twice returns the same KUID with <code>already_registered: true</code> , rather than a second identity. Refuses: under 18, a closed LGA, a malformed phone, a foreign number, a weak password, an unaccepted privacy notice.
GET /v1/public/athletes/{kuid}	PUBLIC	Name, sport, position, LGA, state, year, age, verification state. Never the phone number, never the date of birth — age is derived. A malformed KUID and a well-formed one that does not exist give the same 404, so the register cannot be walked. The <code>?s=</code> signature, when valid, sets <code>issued_by_kafriada</code> .
GET .../{kuid}/qr.svg	PUBLIC	The signed QR, rendered on the server as SVG. Error correction M, so a card that has spent a season in a pocket still scans. Cached 24 hours — a KUID's QR never changes.
GET /v1/public/lgas	PUBLIC	All 27 with their open state. Closed ones are returned rather than hidden, so the form can say "registration opens soon" instead of leaving someone unable to find their town.
GET /healthz	PUBLIC	Liveness only. Deliberately reveals no version, environment or dependency state.

WHY EVERY ROUTE SAYS WHO MAY CALL IT

Authorisation is an argument on the route, not a comment. `Public()` requires a written reason, so "anyone may scan a QR code" reads differently from a route somebody forgot to protect. There is no default — omitting the declaration is a visible mistake rather than a silent opening.

The guarantees, and how each is enforced

Every claim below is enforced by something you can point at, not by good intentions.

GUARANTEE	ENFORCED BY	WHY IT IS DONE THAT WAY
Nobody gets two identities	Unique index on the phone number, plus normalisation in front of it	A constraint in code can be bypassed by another code path. A unique index cannot. The normalisation matters just as much — a constraint on inconsistently-written values does nothing.
A KUID never changes	Database trigger on the athletes table	The application offers no way to change it, and this makes that true of the database too — however the UPDATE was issued.
No two athletes share a KUID	Two unique indexes plus an atomic counter	One index covers the printed string, the other covers (state, year, serial) which is what the counter allocates. If the counter were ever changed wrongly, the second index refuses the write instead of letting two people share an identity.
History cannot be rewritten	Grants, plus a trigger that stops the owner too	Grants restrain the application. They do not restrain the account that owns the table, so the trigger does. Proved by test, not asserted.
The app cannot touch money	Default privileges in the money schema	The connection either has the grant or it does not. No application logic is involved and none can be talked around.
No money can leave	A CI grep that fails the build	No withdrawal, payout or peer-to-peer path may exist — not disabled, absent. This is what keeps KAFRIADA out of banking regulation, and it must survive staff turnover.
A retry never double-charges or double-mints	Unique constraint plus a caught violation	Users are on 2G. Requests time out and get retried. The retry path returns the existing identity rather than creating a second.
Registration cannot be opened by accident	The <code>is_live</code> flag, checked in the service	Checked in the service rather than only in the UI, because closing a wave has to actually stop registrations, not merely remove a button.
A leaked backup reveals no live sessions	Sessions stored as digests	The table holds values that cannot be replayed as a cookie.
Response timing reveals nothing	A real hash verified on the unknown-user path	Otherwise the time a sign-in takes tells an attacker which phone numbers are registered — itself personal data in a register of athletes.

The registration transaction, statement by statement

This ordering is the most performance-critical thing in the system and the reasoning is worth understanding.

validate everything that can be rejected
hash the password ~80ms
BEGIN

load the LGA, check it is open
insert the user account
ONE statement: mint the serial
+ create the athlete
+ write the career event
insert the audit row
COMMIT

no database work yet
OUTSIDE the transaction – the whole trick

no lock
unique phone; a duplicate ends it here
← counter row LOCKED from here

← lock released

Hashing inside the transaction would hold the lock ~80ms → ~12 registrations/sec

Hashing outside, three statements collapsed to one → ~330/sec in-region

Measured for real against Supabase from a laptop on another continent:

four round trips of lock hold 8 of 30 registrations cancelled

one round trip of lock hold 0 of 60 cancelled

WHAT DID NOT HAPPEN, IN EITHER MEASUREMENT

No duplicate KUID was ever issued. Under contention the system **shed load rather than handing two people one identity**. That is the correct failure mode and it is what the design was aiming for.

08 The tests, and what each one proves

112 passing. Not coverage for its own sake — each file below corresponds to a way the system could be wrong that nobody would notice by using it.

FILE	COUNT	WHAT IT SETTLES
test_security_primitives.py	~40	Passwords verify and fail correctly, a corrupted hash fails closed rather than raising, an oversized password is rejected before hashing, the unknown-user path costs the same as a real one. Signatures are bound to their KUID so one cannot be lifted onto another card, and rotation keeps old cards working . A re-serialised webhook body fails its signature — the exact bug that presents as "Paystack is sending bad signatures" and costs a day.
test_settings_refuses_insecure_config.py	~15	Production refuses to start with docs on, DEBUG logging, a wildcard host, open CORS, a test payment key, or app and money sharing a database role. Every problem is reported at once, because fixing a misconfiguration one restart at a time uses up a deploy window.
test_identity_anchor.py	~45	The most important single test in the suite: thirteen ways of writing one phone number all produce one stored value . If that fails, the unique constraint is silently doing nothing. Also: ambiguous input refused rather than guessed, a foreign number refused rather than stripped, KUIDs parsed from what people actually type, all 27 LGA codes produce valid KUIDs.
test_audit_log_is_immutable.py	9	Runs against a real PostgreSQL, because these are claims about the database and nothing else can settle them. The application can write an audit row and cannot change, delete or truncate one. The owner cannot either. The app role holds no write privilege in the money schema.
test_registration_burst.py	3	Real threads, real transactions, real database. No duplicate identity, no gap in the serials, and a retried registration returns the existing KUID. Sizes itself to the environment by measuring the round trip first, because queueing more threads than the link can drain proves only that PostgreSQL cancels statements after fifteen seconds.

Decisions made, and why

DECISION	ALTERNATIVE REJECTED	REASON
Our own login, not Supabase Auth	Supabase Auth in the browser	Supabase gives the phone a pass that stays valid up to an hour <i>after</i> access is removed. Coordinators handle cash on shared phones and need cutting off instantly. We had already built argon2id and database-backed sessions, so the saving was small and the cost was real.
Synchronous database access	Async SQLAlchemy	An accidental <code>await</code> inside a held lock turns a 2ms lock into a 200ms one and collapses registration for the whole state. Recorded as ADR 0001.
A counter row, not a sequence	A PostgreSQL sequence	A sequence never blocks but leaves gaps on rollback. An unexplained gap in a printed public identity number destroys confidence in a register in its first month. We took measured contention over a question we would answer forever.
Phone number as the identity anchor	NIN	Storing NIN without verifying it carries the compliance duty with none of the benefit, and verifying it costs money nobody budgeted. NIN linkage is Phase 2.
A separate Supabase bootstrap	Running the standard one	The standard script strips privileges from PUBLIC, which would break Supabase's own dashboard and API. Our isolation comes from three schemas we own, not from the state of the shared area.
Verification state is not on the athlete row	Denormalising it for speed	It would give the system two answers to "is this athlete verified" — which is the exact question the product is sold on.
No DELETE granted by default	Standard read/write grants	The default for a register of people and money is that nothing is deleted. Where a row genuinely must go, that migration grants DELETE on that one table with the reason written down. So far only expired sessions qualify.

What is not built yet

Stated plainly, so nothing in a presentation overstates what exists.

MISSING	STAGE	WHAT IT BLOCKS
Every screen	1	There is no user interface at all. The API works and is proved; nothing yet renders it. This is the immediate next job.
Sign-in and sessions	1	Registration creates an account and a password hash, but there is no way to log in with it yet. <code>Requires()</code> refuses every protected route until this lands.
SMS one-time codes	1	Phone numbers are collected and normalised but not yet confirmed. Needs a Termii key.
The permission matrix test	1	Every role against every route, with a second tenant. Cannot be written until routes need permissions.
Payments, verification, photos	2	The 4% conversion gate. Nothing about money exists beyond the privilege boundary waiting for it.
Clubs and rosters	2	The "81 clubs" gate.
Background worker	2	Reconciliation, expiry, the nightly integrity check.
Transfers and the news feed	Slice 2	Deliberately after launch — neither is a gate metric.
Deploy pipeline, Cloudflare, backups	3	Everything runs locally against a live database. Nothing is deployed.

11 Known debt and open risks

Written down so none of it is discovered later.

1 — SUPABASE CREDENTIALS ARE STILL NOT ROTATED

The database password, service role key and JWT secret were pasted into a chat transcript and remain live. Low risk today — no real athlete has registered — and unacceptable the moment one does. Two minutes in Project Settings.

2 — 115 TEST ATHLETES ARE IN THE LIVE DATABASE

Left by the burst test, and they cannot be removed by the application because career events are append-only and athletes carry no DELETE grant. That is the system behaving correctly. Before real registrations begin, either accept them as seeded data or rebuild the database from migrations — which takes about a minute and is a good rehearsal for the restore drill.

3 — THE LGA CODES NEED SIGN-OFF

Twenty-six of the twenty-seven three-letter codes were chosen by me. They are printed inside every KUID issued in that LGA and can never change. **The CEO and the state coordinator should approve the list before the first registration drive.**

4 — A SECURITY BUG THAT HAD ALREADY BEEN WRITTEN

The client IP went straight into an `inet` column. A caller sending `cf-connecting-ip:nonsense` would have failed the audit insert inside the registration transaction and taken registration down for everybody, at no cost to themselves. Found while testing, fixed, and worth recording because it is the shape of bug to watch for: **attacker-controlled headers reaching typed database columns.**

5 — SMALLER ITEMS

- Python 3.13 is used because it is what is installed; the plan said 3.12. Production must match whichever is chosen.
- Registration runs at 1.2/sec *from this laptop* — that measures the distance to Frankfurt, not the code. In-region it projects to roughly 330/sec. Do not quote the low number without the explanation.
- The `decobwebs` GitHub token has expired; pushes work through Windows Credential Manager as `afss-git`.
- Repository privacy was confirmed verbally but could not be checked programmatically.
- Paystack verification and the Termii sender ID are still not started. Neither blocks the build; both block launch, and the CAC registration question behind Paystack is still unchecked.